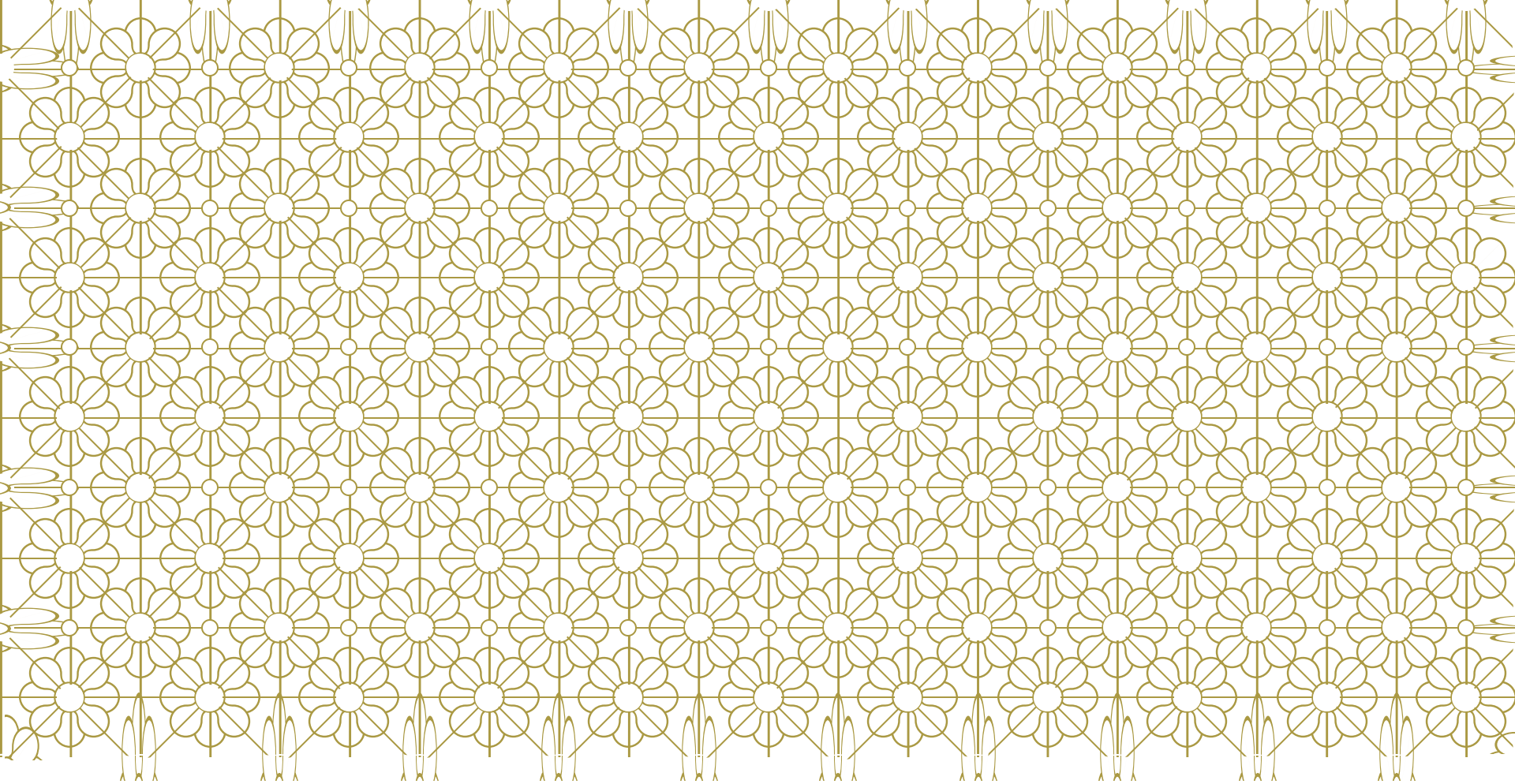
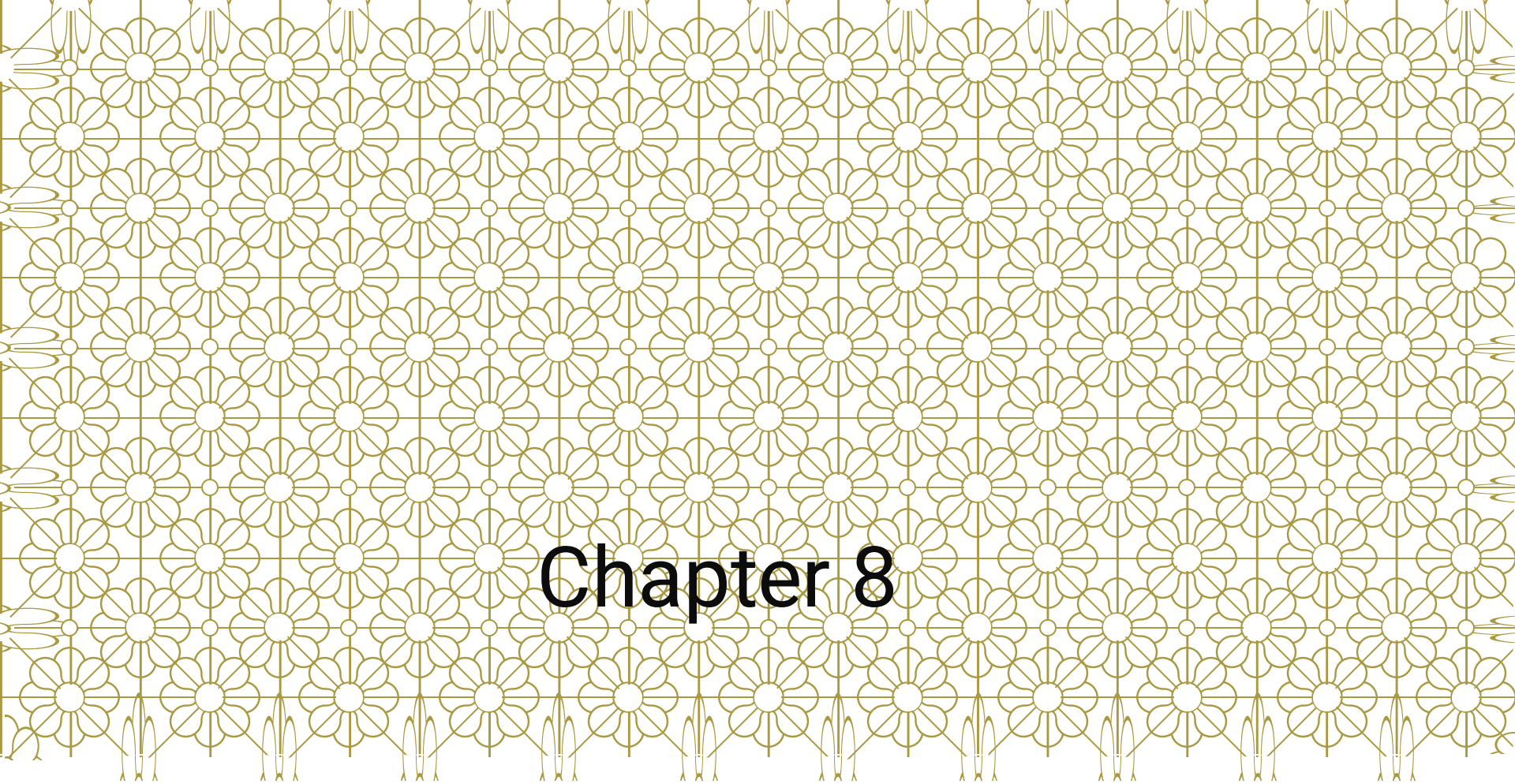




Algorithms analysis and Design

By : Dr. Basem
Alijla



Chapter 8

Dynamic Programming

OUTLINE

- Change-making problem
- Coin-collecting problem

The Knapsack Problem and Memory Functions

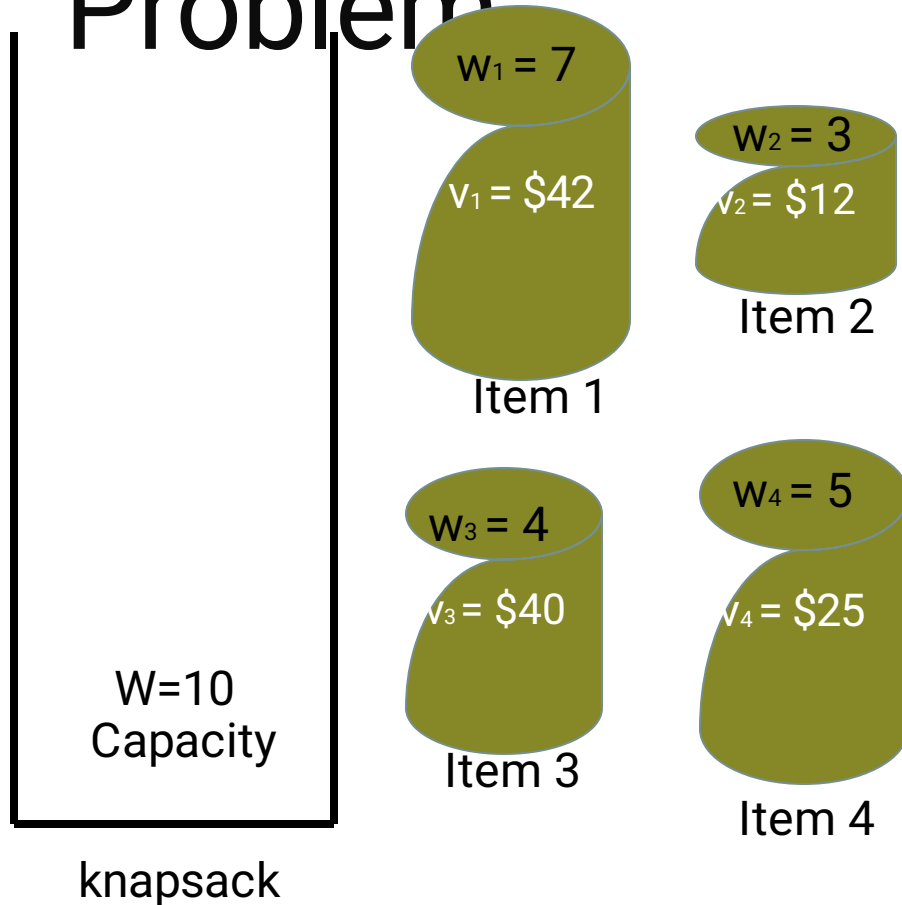
Optimal Binary Search Tree

Warshall's and Floyd's Algorithms

The Knapsack Problem and Memory Functions

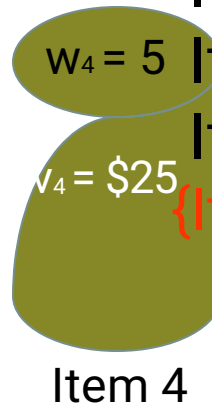
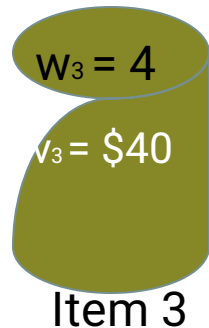
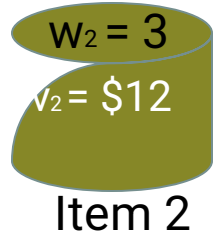
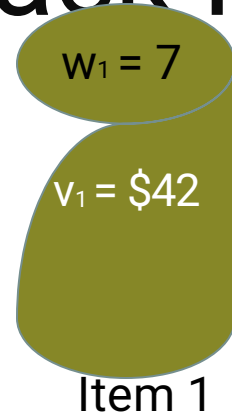
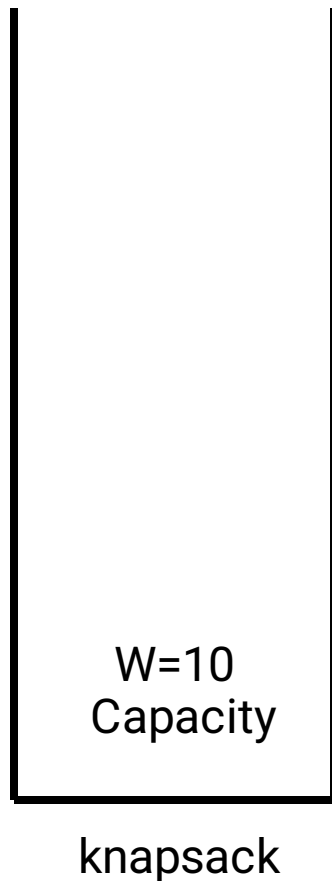
- Given n items of weights $w_1, w_2, \dots, w_n$ and values $v_1, v_2, \dots, v_n$ and a knapsack of capacity W .
- Find the most valuable subset of the items that fit into the knapsack.

Brute force for Knapsack Problem



subset	weight	value
$\emptyset$	0	\$0
{1}	7	\$42
{2}	3	\$12
{3}	4	\$40
{4}	5	\$25
{1,2}	10	\$54
{1,3}	11	!feasible
{1,4}	12	!feasible
{2,3}	7	\$52
{2,4}	8	\$37
{3,4}	9	\$65
{1,2,3}	14	!feasible
{1,2,4}	15	!feasible
{1,3,4}	16	!feasible
{2,3,4}	12	!feasible
{1,2,3,4}	19	!feasible

Greedy Algorithm for Knapsack Problem



How about taking items in decreasing order of value/weight?

Item3: \$10/unit

Item1: \$6/unit

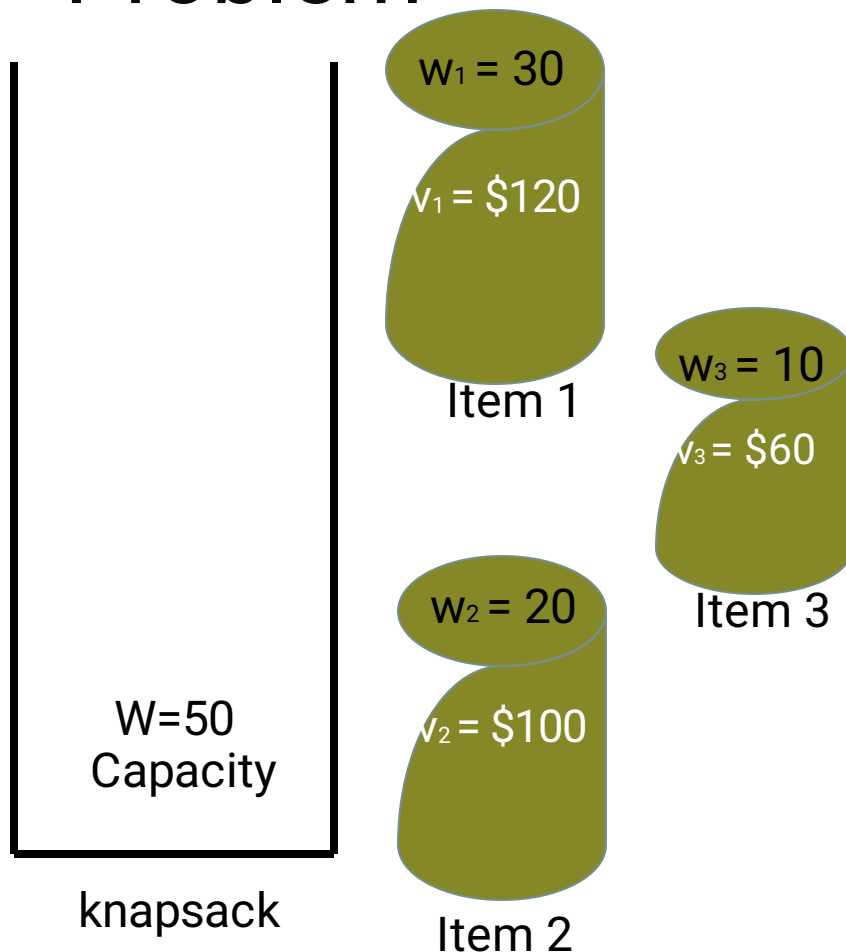
Item4: \$5/unit

Item2: \$4/unit

{Item3, Item4} = \$40+\$25 = \$65

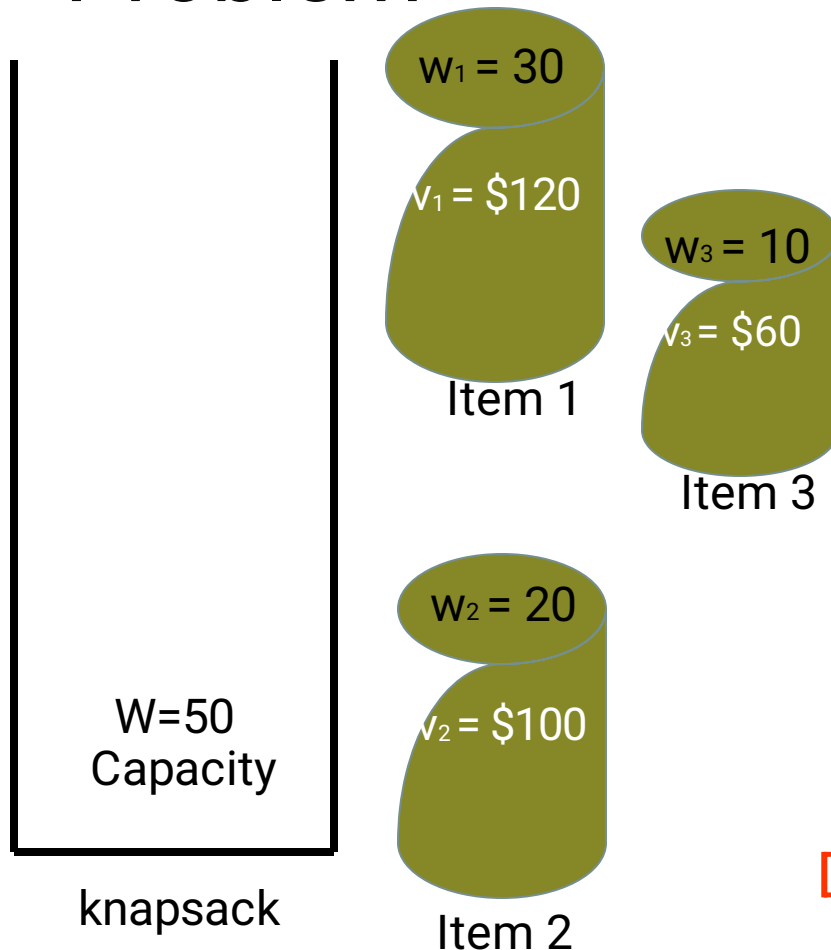
Works for this example!

Greedy Algorithm for Knapsack Problem



subset	weight	value
$\emptyset$	0	\$0
{1}	30	\$120
{2}	20	\$100
{3}	10	\$60
{1,2}	50	\$220
{1,3}	40	\$180
{2,3}	30	\$160
{1,2,3}	60	!feasible

Greedy Algorithm for Knapsack Problem



How about taking items in decreasing order of value/weight?

Item3: \$6/unit

Item 2: \$5/unit

Item1: \$4/unit

{Item3, Item2} = \$60+\$100 = \$160

{Item3, Item1} = \$60+\$120 = \$180

{Item2, Item1} = \$100+\$120 = \$220

Doesn't work for this example

The Knapsack Problem and Memory Functions

- Designing a dynamic programming algorithm for the knapsack problem:
- Consider instance defined by first i items: $1 \leq i \leq n$ with :
 - weights $w_1, \dots, w_i$, values $v_1, \dots, v_i$ and
 - knapsack capacity j ($1 \leq j \leq W$).
- Let $F[i,j]$ be optimal value of such instance.

The Knapsack Problem and Memory Functions

- We can divide all the subsets of the first i items that fit the knapsack of capacity j into two categories: do not include the i th item and those that do.
- The optimal value of the subset that do not include the i th item is $F(i-1, j)$.
- The optimal value of the subset that includes the i th item, and the first $i - 1$ items that fits into the knapsack of capacity $(j - w_i) \geq 0$. $v_i + F(i-1, j - w_i)$.

The Knapsack Problem and Memory Functions

- the value of an optimal solution among all feasible subsets of the first I items is the maximum of these two values.
- $\text{MAX} \{ F(i-1, j) \text{ and } v_i + F(i-1, j - w_i) \}$
- if the i th item does not fit into the knapsack, the value of an optimal subset selected from the first i items is the same as the value of an optimal subset selected from the first $i - 1$ items.
- $F(i-1, j)$

The Knapsack Problem and Memory Functions

- $F(i, j) = \begin{cases} \max \{F(i-1, j), v_i + F(i-1, j - w_i)\} & \text{if } j - w_i \geq 0, \\ F(i-1, j) & \text{if } j - w_i < 0 \end{cases}$
- $F(0, j) = 0$ for $j \geq 0$ and $F(i, 0) = 0$ for $i \geq 0$.

EXAMPLE 1: Solve Ksp using DP

- Let us consider the instance given by the following data:

Item	Weight	Value
1	2	\$12
2	1	\$10
3	3	\$20
4	2	\$15

Capacity
 $W = 5$

EXAMPLE 1: Solve Ksp using DP

Item	Weight	Value
1	2	\$12
2	1	\$10
3	3	\$20
4	2	\$15

$W = 5$

i	Capacity					
	0	1	2	3	4	5
0						
1						
2						
3						
4						

EXAMPLE 1: Solve Ksp using DP

Ite	Weight	Value
1	2	\$12
2	1	\$10
3	3	\$20
4	2	\$15

W = 5

i	Capacity					
	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0					
2	0					
3	0					
4	0					

EXAMPLE 1: Solve Ksp using DP

Ite	Weight	Value
1	2	\$12
2	1	\$10
3	3	\$20
4	2	\$15

W = 5

$F(i, j)$

$\max\{F(i-1, j), v_i + F(i-1, j - w_i)\}$ if $j - w_i \geq 0$.

$F(i-1, j)$

if $j - w_i < 0$.

i	Capacity					
	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	12	12	12	12
2	0					
3	0					
4	0					

EXAMPLE 1: Solve Ksp using DP

Ite	Weight	Value
1	2	\$12
2	1	\$10
3	3	\$20
4	2	\$15

W = 5

$F(i, j)$

$\max\{F(i-1, j), v_i + F(i-1, j - w_i)\}$ if $j - w_i \geq 0$.

$F(i-1, j)$

if $j - w_i < 0$.

i	Capacity					
	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	12	12	12	12
2	0					
3	0					
4	0					

Max { 0, 10 + 0 }

EXAMPLE 1: Solve Ksp using DP

Ite	Weight	Value
1	2	\$12
2	1	\$10
3	3	\$20
4	2	\$15

W = 5

$F(i, j)$

$\max\{F(i-1, j), v_i + F(i-1, j - w_i)\}$ if $j - w_i \geq 0$.

$F(i-1, j)$

if $j - w_i < 0$.

i	Capacity					
	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	12	12	12	12
2	0	10				
3	0					
4	0					

$\text{Max}\{12, 10 + 0\}$

EXAMPLE 1: Solve Ksp using DP

Ite	Weight	Value
1	2	\$12
2	1	\$10
3	3	\$20
4	2	\$15

W = 5

$F(i, j)$

$\max\{F(i-1, j), v_i + F(i-1, j - w_i)\}$ if $j - w_i \geq 0$.

$F(i-1, j)$

if $j - w_i < 0$.

i	Capacity					
	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	12	12	12	12
2	0	10	12			
3	0					
4	0					

Max { 12, 10 + 12 }

EXAMPLE 1: Solve Ksp using DP

Ite	Weight	Value
1	2	\$12
2	1	\$10
3	3	\$20
4	2	\$15

W = 5

$$F(i, j) = \begin{cases} \max\{F(i-1, j), v_i + F(i-1, j - w_i)\} & \text{if } j - w_i \geq 0. \\ F(i-1, j) & \text{if } j - w_i < 0. \end{cases}$$

i	Capacity					
	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	12	12	12	12
2	0	10	12	22		
3	0					
4	0					

EXAMPLE 1: Solve Ksp using DP

Ite	Weight	Value
1	2	\$12
2	1	\$10
3	3	\$20
4	2	\$15

W = 5

$$F(i, j) = \begin{cases} \max\{F(i-1, j), v_i + F(i-1, j - w_i)\} & \text{if } j - w_i \geq 0. \\ F(i-1, j) & \text{if } j - w_i < 0. \end{cases}$$

i	Capacity					
	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	12	12	12	12
2	0	10	12	22	22	22
3	0					
4	0					

EXAMPLE 1: Solve Ksp using DP

Ite	Weight	Value
1	2	\$12
2	1	\$10
3	3	\$20
4	2	\$15

W = 5

$$F(i, j) = \begin{cases} \max\{F(i-1, j), v_i + F(i-1, j - w_i)\} & \text{if } j - w_i \geq 0. \\ F(i-1, j) & \text{if } j - w_i < 0. \end{cases}$$

i	Capacity					
	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	12	12	12	12
2	0	10	12	22	22	22
3	0	10	12	22	30	32
4	0					

EXAMPLE 1: Solve Ksp using DP

Item	Weight	Value
1	2	\$12
2	1	\$10
3	3	\$20
4	2	\$15

W = 5

$F(i, j)$

$\max\{F(i-1, j), v_i + F(i-1, j - w_i)\}$ if $j - w_i \geq 0$.

$F(i-1, j)$

if $j - w_i < 0$.

i	Capacity					
	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	12	12	12	12
2	0	10	12	22	22	22
3	0	10	12	22	30	32
4	0	10	15	25	30	37

EXAMPLE 1: Solve Ksp using DP

Ite	Weight	Value
1	2	\$12
2	1	\$10
3	3	\$20
4	2	\$15

find the composition of an optimal subset by backtracing the computations of this entry in the table.

The time efficiency and space efficiency of this algorithm are both in $O(nW)$

i	Capacity					
	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	12	12	12	12
2		F(1, 2) > F(0, 2), item 1 is included				22
3	0	10	F(2, 3) > F(1, 3), item 2 is included			32
4	0	1	F(3, 3) = F(2, 3), item 3 not included			37

F(4, 5) > F(3, 5), item 4 is included

Memory functions

- The direct top-down approach to finding a solution to such a recurrence is very inefficient ($O(2^n)$) or worse)
- The classic dynamic programming approach works bottom up it fills a table with solutions to all smaller subproblems. each of them is solved only once.
- some of these smaller subproblems are often not necessary for getting a solution to the problem given.

Memory functions

- Need a method based on Memory functions to solves only subproblems that are necessary and does so only once.
- This method solves a given problem in the top-down manner but, in addition, maintains a table of the kind that would have been used by a bottom-up dynamic programming algorithm.
- Initially, all the table's entries are initialized with -1 to indicate that they have not yet been calculated

Memory functions Knapsack algorithms

- ALGORITHM MFKnapsack(i, j)
- if $F[i, j] < 0$
 - if $j < \text{Weights}[i]$
 - value MFKnapsack($i - 1, j$)
 - else
 - value $\max(\text{MFKnapsack}(i - 1, j), \text{Values}[i] + \text{MFKnapsack}(i - 1, j - \text{Weights}[i]))$
- $F[i, j] = \text{value}$
- return $F[i, j]$

EXAMPLE 1: Solve Ksp MFKnapsack

Weight	2	2	3	4
--------	---	---	---	---

Value	1	2	3	4
	12	10	20	15

Only 11 out of 20 nontrivial values (i.e., not those in row 0 or in column 0) have been computed

i	Capacity					computed	5
	0	1	2	3	4		
0	0	0	0	0	0	0	0
1	0	-1	-1	-1	-1	-1	-1
2	0	-1	-1	-1	-1	-1	-1
3	0	-1	-1	-1	-1	-1	-1
4	0	-1	-1	-1	-1	-1	-1

MFKnapsack(4,5)
28

OUTLINE

- Change-making problem
- Coin-collecting problem

The Knapsack Problem and Memory Functions

Optimal Binary Search Tree

Warshall's and Floyd's Algorithms